GitHub

Introdução

GitHub

- GitHub - Profº Leandro
- https://www.youtube.com/playlist?list=PLTLAlheiUm5EohFD_1TDCpYkEvLKNWCV8

- GitHub Desktop
- https://www.youtube.com/watch?v=EGmzAs1G0z0&t=3s

PHP

Preparando o ambiente

Preparando o ambiente

Preparando o ambiente

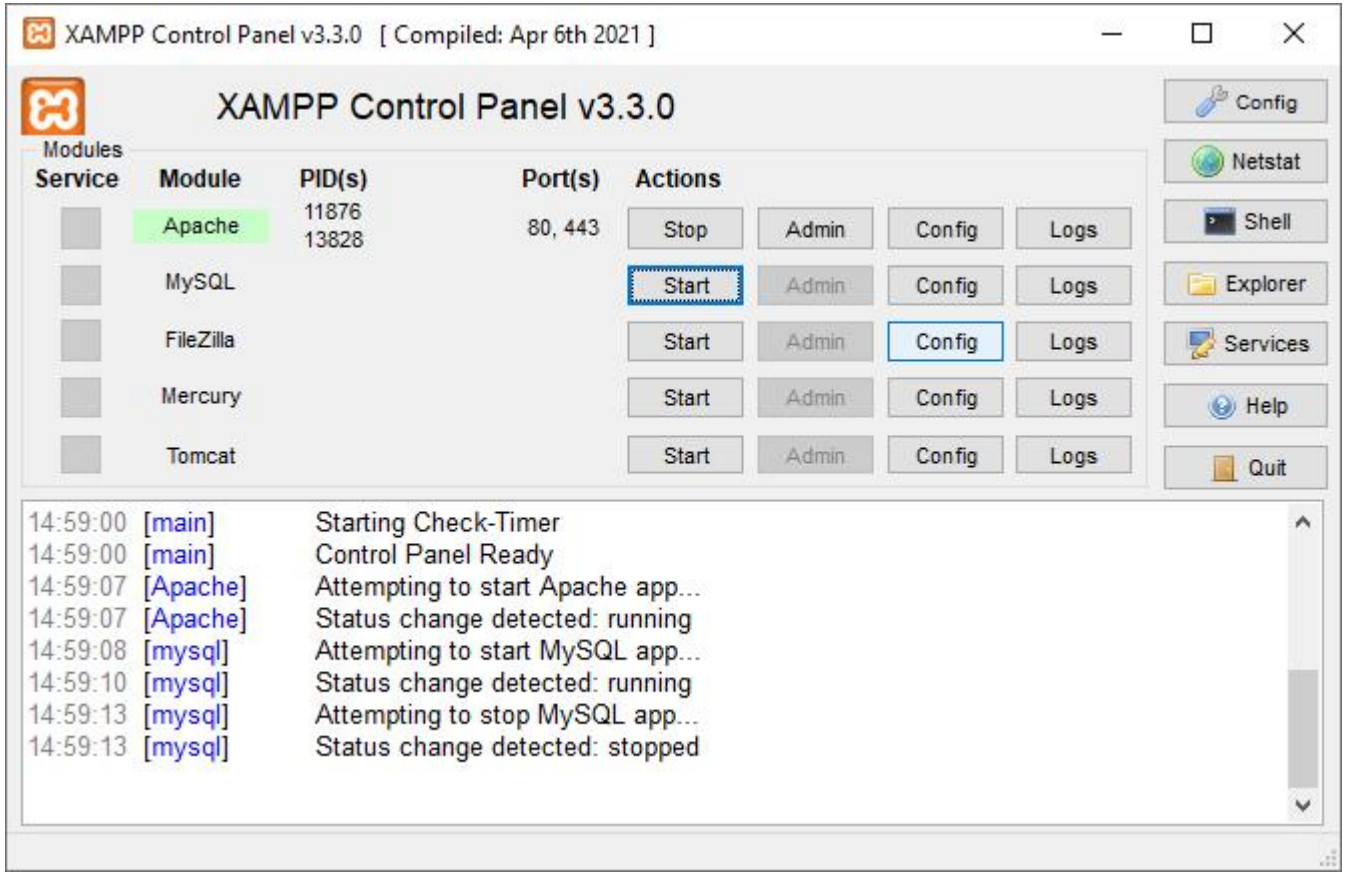
- VS Code
  - https://code.visualstudio.com/

- XAMPP com PHP 8.2 ou superior
  - https://www.apachefriends.org/pt_br/index.html

- Documentação PHP
  - https://www.php.net/

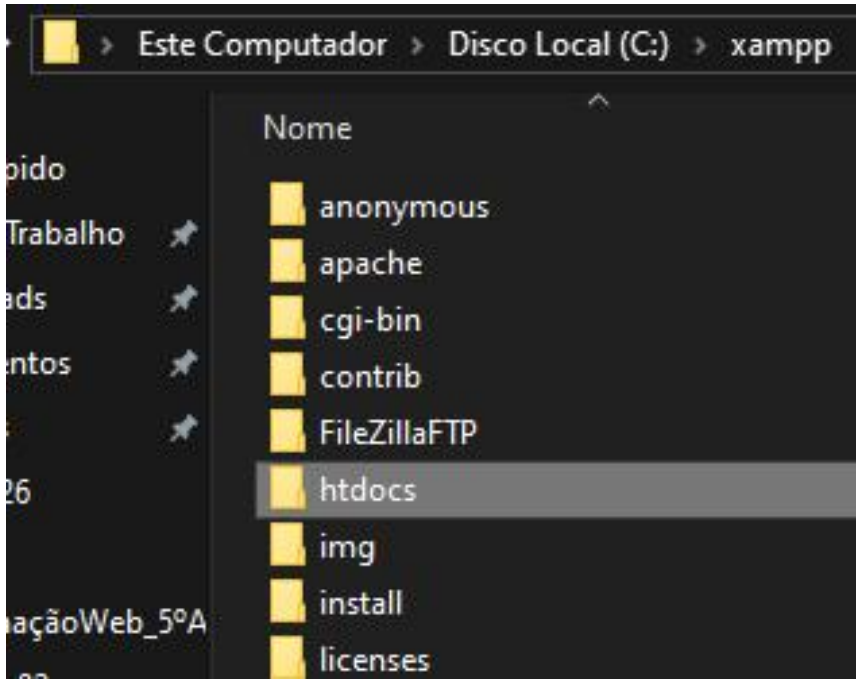
Executar XAMPP

- Executar o APACHE



Conde criar os arquivo / projetos PHP

- C:/xampp/htdocs



PHP

Introdução

PHP

- Foi criada em 1994 por Rasmus Lerdorf, inicialmente para gerenciar páginas pessoais, mas evoluiu para uma das linguagens mais usadas na web.

- Originalmente, a sigla PHP significava **Personal Home Page**.

- Hoje chamada de **“PHP: Hypertext Preprocessor”**.

PHP

- É uma linguagem de programação interpretada, utilizada principalmente para o desenvolvimento de sites e aplicações web dinâmicas.

- Interpretada significa que o código é executado linha por linha por um programa chamado interpretador, sem precisar ser transformado antes em um arquivo executável.

PHP

Primeiros códigos

Estrutura básica do PHP

- Todo código PHP deve começar e terminar com as tags:

```
<?php
// código PHP aqui
// Tudo que estiver dentro dessas tags será interpretado pelo servidor
?>
```

PHP com HTML

- O PHP pode ser misturado com HTML.
- O PHP é executado no servidor e o resultado aparece no navegador.

```
<!DOCTYPE html>
  <html>
    <body>
        <h1>Minha página</h1>
         <?php
              echo "Olá, mundo!";
          ?>
  </body>
</html>
```

Comando de saída (echo)

- O echo é usado para mostrar informações na tela.
- Pode mostrar textos, números e resultados de cálculos.

```
<?php
    echo "Olá, mundo!";
?>
```

Variáveis em PHP

- Variáveis começam com **$**.
- Não é necessário declarar o tipo da variável.

```
<?php
$nome = "Maria";
$idade = 25;

echo $nome;
echo $idade;
?>
```

Operadores Aritméticos

```
<?php
    $a = 10;
    $b = 5;

    echo $a + $b; // soma
    echo $a - $b; // subtração
    echo $a * $b; // multiplicação
    echo $a / $b; // divisão
    echo $a % $b; // resto da divisão
?>
```

Operadores de Atribuição

- Servem para atualizar valores de variáveis:

```
<?php
        $x = 10;

        $x += 5; // x = x + 5
        $x -= 3; // x = x - 3
        $x *= 2; // x = x * 2
        $x /= 4; // x = x / 4
        $x %= 3; // resto da divisão

        echo $x;
?>
```

Concatenação em PHP (Operador ponto .)

- O operador . (ponto) é usado para juntar textos (strings).

```
<?php
$nome = "João";

echo "Olá, " . $nome;
?>
```

Resultado: Olá, João

Strings com Aspas Simples (' ')

- As aspas simples tratam o conteúdo como texto literal.

```
<?php
$nome = "João";

echo 'Olá, $nome';
?>
```

Resultado: Olá, $nome (não substitui a variável)

Strings com Aspas Duplas (" ")

- As aspas duplas permitem interpretar variáveis.

```
<?php
$nome = "João";

echo "Olá, $nome";
?>
```

Resultado: Olá, João

Interpolação de Variáveis

- Interpolação é quando o PHP substitui a variável pelo valor dentro da string.
- Forma recomendada com { }:

```
<?php
$nome = "João";

echo "Olá, {$nome}";
?>
```

Garante melhor leitura e evita erros em textos maiores.

Diferença Importante

```php
<?php
    $nome = "Ana";

    echo "Nome: $nome";   // funciona
    echo 'Nome: $nome';   // não funciona
?>
```

Lista de Exercícios

PHP

Estruturas Condicionais em PHP

Operadores de Comparação

Operador	Significado	Exemplo	Resultado
>	Maior que	5 > 3	true
<	Menor que	2 < 4	true
>=	Maior ou igual	5 >= 5	true
<=	Menor ou igual	3 <= 2	false
==	Igual (valor)	5 == "5"	true
===	Igual (valor e tipo)	5 === "5"	false
!=	Diferente	5 != 3	true
!==	Diferente (valor ou tipo)	5 !== "5"	true

Diferença importante: == vs ===

- == compara apenas o valor
- === compara valor e tipo

```
5 == "5"   // true (mesmo valor)

5 === "5"  // false (tipos diferentes)
```

Operadores Lógicos

- Servem para combinar condições.

Operador	Nome	Significado
&&	e	Todas as condições devem ser verdadeiras para que a condição seja verdadeira
||	ou	Todas as condições devem ser falsas para que a condição seja falsa
!	negação	Inverte o valor lógico

Estrutura IF

```
if (condicao) {

    // código executado se a condição
    // for verdadeira

}
```

```
$idade = 18;

if ($idade >= 18) {
    echo "Maior de idade";
}
```

IF com ELSE

```
if (condicao) {
    // verdadeiro
} else {
    // falso
}
```

```
$idade = 16;

if ($idade >= 18) {
    echo "Maior de idade";
} else {
    echo "Menor de idade";
}
```

IF com ELSEIF

```
if (condicao1) {
    // código
} elseif (condicao2) {
    // código
} else {
    // código
}
```

```
$nota = 7;

if ($nota >= 9) {
    echo "Excelente";
} elseif ($nota >= 6) {
    echo "Aprovado";
} else {
    echo "Reprovado";
}
```

Operador Ternário

- Uma forma resumida de escrever um if/else.

```
(condicao) ? valor_se_verdadeiro :
valor_se_falso;
```

```
$idade = 20;

$resultado = ($idade >= 18) ? "Maior de
idade" : "Menor de idade";

echo $resultado;
```

Quando usar o Ternário ou if-else?

- Utilize o operador ternário:

  - Para decisões simples
  - Para reduzir código
  - Uma única instrução

- Utilize if-else:

  - Há muitas condições
  - Código grande
  - Há muitas instruções

SWITCH

- Utilizde switch quando existir várias opções baseadas no mesmo valor.

```
switch ($variavel) {
    case valor1:
        // código
        break;

    case valor2:
        // código
        break;

    default:
        // código padrão
}
```

```
$dia = 3;

switch ($dia) {
    case 1:
        echo "Domingo";
        break;

    case 2:
        echo "Segunda";
        break;

    case 3:
        echo "Terça";
        break;

    default:
        echo "Dia inválido";
}
```

Importante sobre o SWITCH

- O break evita que outros casos sejam executados
- O default funciona como o else

Operador && (E)

- Só entra no if se todas as condições forem verdadeiras

```
$idade = 20;
$temCarteira = true;

if ($idade >= 18 && $temCarteira == true) {
    echo "Pode dirigir";
} else {
    echo "Não pode dirigir";
}
```

Explicação:
- Tem 18 anos OU mais e tem carteira
- **Resultado:** Pode dirigir

Operador || (OU)

- Entra no if se pelo menos uma condição for verdadeira

```
$dia = "sábado";

if ($dia == "sábado" || $dia == "domingo") {
    echo "Final de semana!";
} else {
    echo "Dia de semana";
}
```

Explicação:
- É sábado? ou é domingo?
- **Resultado:** Final de semana

Operador ! (NÃO / NEGAÇÃO)

- Inverte o valor lógico

```
$logado = false;

if (!$logado) {
    echo "Usuário não está logado";
    // redireciona para página de erro
}
```

Explicação:
- $logado é falso
- !false = verdadeiro
- **Resultado:** Usuário não está logado

Exemplo combinado (&& + || + !)

```
$idade = 17;
$temPermissao = true;
$bloqueado = false;

if (($idade >= 18 || $temPermissao == true) && !$bloqueado) {
    echo "Acesso liberado";
} else {
    echo "Acesso negado";
}***
```

Explicação:

- Idade ≥ 18? (não)
- Tem permissão? (sim)
  (uma das duas é verdadeira → OK)

Está bloqueado?
- !bloqueado = verdadeiro

- **Resultado final:** Acesso liberado

DICA: Use parênteses para organizar melhor a ordem de execução das expressões.

Lista de Exercícios

PHP

Estrutura de Repetição

O que são Estruturas de Repetição?

- São comandos que permitem repetir um bloco de código enquanto uma condição for verdadeira.

- Exemplo do dia a dia:

  “Enquanto não terminar, continue estudando.”

WHILE

```
while (condicao) {
    // código que será repetido
}
```

```
$contador = 1;

while ($contador <= 5) {
    echo "Número: $contador <br>";
    $contador++;
}
```

Explicação:

- Começa em 1
- Repete até 5
- Incrementa de 1 em 1

DO/WHILE

- Executa pelo menos uma vez, mesmo que a condição seja falsa

```
do {
    // código
} while (condicao);
```

```
$contador = 1;

do {
    echo "Número: $contador <br>";
    $contador++;
} while ($contador <= 5);
```

FOR

- Quando usar?
  - Quando você sabe quantas vezes o loop vai executar

```
for (inicializacao; condicao; incremento) {
    // código
}
```

```
for ($i = 1; $i <= 5; $i++) {
    echo "Número: $i <br>";
}
```

Explicação:

$i = 1 → começa em 1
$i <= 5 → vai até 5
$i++ → soma 1 a cada repetição

FOREACH

- Quando usar?
  - Quando você quer percorrer arrays (listas)

```
foreach ($array as $valor) {
    // código
}
```

```
$nomes = ["Ana", "Carlos", "Maria"];

foreach ($nomes as $nome) {
    echo "Nome: $nome <br>";
}
```

Comparação Geral

- while

  Quando não sabe quantas repetições

- do/while

  Quando precisa executar ao menos 1 vez

- for

  Quando sabe o número de repetições

- foreach

  Para percorrer arrays

Lista de Exercícios